

Online Scalable Gaussian Processes with Conformal Prediction: A Rigorous Derivation Companion with the implementing code

Derivation notes on Xu, Lu & Giannakis (ICASSP 2025; arXiv:2410.05444)

Scope and conventions. This document follows the paper section by section and fills in, with rigour, the derivations the paper compresses. Displayed equations are transcribed as in the manuscript; the surrounding text is a paraphrase, with only short quoted phrases. Each result is paired with the function in `Ensemble Gaussian Process.py` that implements it. In the code, `sigma_f` denotes the kernel magnitude σ_θ , `sigma_n` the observation-noise standard deviation σ_n , and the array `V` stores the D sampled spectral frequencies, one per column. Throughout, $\{(\mathbf{x}_t, y_t)\}_{t \geq 1}$ is a stream in $\mathbb{R}^d \times \mathbb{R}$, and $\mathcal{D}_t = \{\mathbf{x}_\tau, y_\tau\}_{\tau \leq t}$.

Contents

1	Setup and the observation model	1
2	Gaussian-process regression (§II.A)	2
3	Conformal prediction and the coverage guarantee (§II.B)	2
4	Scalable GP by random Fourier features (§III.A)	4
5	Weight-space posterior and online predictive (§III.A)	5
6	The memoryless recursive update (§III.A, eqs. 10–11)	6
7	Online Bayes credible set (§III.A, eq. 9)	6
8	Conformalised GP set and threshold update (§III.B)	7
9	Learning-rate schedules and change-point reset (§III.B)	8
10	Long-run coverage guarantee (§III.B, eq. 15)	8
11	Hyperparameters and experimental setup (§IV)	9
12	Extension in the code: the kernel ensemble	9
13	Equation-to-code map	10

1 Setup and the observation model

The learner sees inputs $\mathbf{x}_t$ and scalar outputs y_t generated by

$$y_\tau = f(\mathbf{x}_\tau) + n_\tau, \quad n_\tau \sim \mathcal{N}(0, \sigma_n^2) \text{ i.i.d.}, \quad (1)$$

with f unknown. Two goals are in tension: (i) a *point* predictor $\hat{y}_t(\mathbf{x})$, and (ii) a *set* predictor $\mathcal{C}_t(\mathbf{x})$ that contains the true y with a prescribed probability $1 - \alpha$. The Gaussian process supplies both under its modelling assumptions; conformal prediction repairs the set’s coverage when those assumptions fail. Sections 2–3 recall the two ingredients; Sections 4–10 build the online scalable predictor and its guarantee.

2 Gaussian-process regression (§II.A)

Place a zero-mean GP prior $f \sim \mathcal{GP}(0, \kappa(\mathbf{x}, \mathbf{x}'))$ with positive-definite kernel κ . For inputs $\mathbf{x}_1, \dots, \mathbf{x}_t$ the vector $\mathbf{f}_t = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_t)]^\top$ is Gaussian, $p(\mathbf{f}_t \mid \mathbf{x}_{1:t}) = \mathcal{N}(\mathbf{f}_t; \mathbf{0}, \mathbf{K}_t)$ with $[\mathbf{K}_t]_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$. Under (1) the labels satisfy $\mathbf{y}_t \mid \mathbf{f}_t \sim \mathcal{N}(\mathbf{f}_t, \sigma_n^2 \mathbf{I}_t)$, so marginally $\mathbf{y}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{K}_t + \sigma_n^2 \mathbf{I}_t)$. Writing $\mathbf{k}_t(\mathbf{x}) = [\kappa(\mathbf{x}_1, \mathbf{x}), \dots, \kappa(\mathbf{x}_t, \mathbf{x})]^\top$, the joint law of the observed labels and a new (noisy) test label is Gaussian (paper eq. 1):

$$\begin{bmatrix} \mathbf{y}_t \\ y \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K}_t + \sigma_n^2 \mathbf{I}_t & \mathbf{k}_t(\mathbf{x}) \\ \mathbf{k}_t^\top(\mathbf{x}) & \kappa(\mathbf{x}, \mathbf{x}) + \sigma_n^2 \end{bmatrix}\right). \quad (2)$$

Lemma 1 (Gaussian conditioning). *If $\begin{bmatrix} \mathbf{a} \\ \mathbf{b} \end{bmatrix} \sim \mathcal{N}(\begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \mathbf{A} & \mathbf{C} \\ \mathbf{C}^\top & \mathbf{B} \end{bmatrix})$ with $\mathbf{A} \succ 0$, then $\mathbf{b} \mid \mathbf{a} \sim \mathcal{N}(\mu_b + \mathbf{C}^\top \mathbf{A}^{-1}(\mathbf{a} - \mu_a), \mathbf{B} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{C})$.*

Proof. Form $\mathbf{z} = \mathbf{b} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{a}$. Then $\text{Cov}(\mathbf{z}, \mathbf{a}) = \mathbf{C}^\top - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{A} = \mathbf{0}$, so $\mathbf{z} \perp \mathbf{a}$ (jointly Gaussian $\Rightarrow$ independence). Hence $\mathbf{b} \mid \mathbf{a}$ has mean $\mu_b + \mathbf{C}^\top \mathbf{A}^{-1}(\mathbf{a} - \mu_a)$ and covariance $\text{Cov}(\mathbf{z}) = \mathbf{B} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{C}$. $\square$

Applying Lemma 1 to (2) with $\mathbf{a} = \mathbf{y}_t$, $\mathbf{b} = y$, $\mathbf{A} = \mathbf{K}_t + \sigma_n^2 \mathbf{I}_t$, $\mathbf{C} = \mathbf{k}_t(\mathbf{x})$, $\mathbf{B} = \kappa(\mathbf{x}, \mathbf{x}) + \sigma_n^2$ gives the predictive pdf (paper eqs. 2–3):

$$p(y \mid \mathcal{D}_t, \mathbf{x}) = \mathcal{N}(y; \hat{y}_t(\mathbf{x}), \sigma_t^2(\mathbf{x})), \quad (3)$$

$$\hat{y}_t(\mathbf{x}) = \mathbf{k}_t^\top(\mathbf{x}) (\mathbf{K}_t + \sigma_n^2 \mathbf{I}_t)^{-1} \mathbf{y}_t, \quad (4)$$

$$\sigma_t^2(\mathbf{x}) = \kappa(\mathbf{x}, \mathbf{x}) - \mathbf{k}_t^\top(\mathbf{x}) (\mathbf{K}_t + \sigma_n^2 \mathbf{I}_t)^{-1} \mathbf{k}_t(\mathbf{x}) + \sigma_n^2. \quad (5)$$

The β -credible set is $K_t^\beta(\mathbf{x}) = [\hat{y}_t(\mathbf{x}) - c_\beta \sigma_t(\mathbf{x}), \hat{y}_t(\mathbf{x}) + c_\beta \sigma_t(\mathbf{x})]$ with $c_\beta = \Phi^{-1}(\frac{1+\beta}{2})$; for $\beta = 95\%$, $c_\beta \approx 2$.

Remark 1 (Cost and its consequence). Equations (4)–(5) require solving with $\mathbf{K}_t + \sigma_n^2 \mathbf{I}_t \in \mathbb{R}^{t \times t}$, an $O(t^3)$ factorisation that grows without bound in the stream. **No function forms $\mathbf{K}_t$** ; Section 4 replaces the kernel by a finite feature map so that the same predictive becomes an $O((2D)^2)$ recursive update.

3 Conformal prediction and the coverage guarantee (§II.B)

Let $s_t(\mathbf{x}, y)$ be a *non-conformity score*: larger means y conforms worse to the fitted model. Given a threshold q_t , the prediction set (paper eq. 4) is

$$\mathcal{C}_t(\mathbf{x}) = \{y \in \mathbb{R} : s_t(\mathbf{x}, y) \leq q_t\}. \quad (6)$$

The classical choice takes q_t to be the $\lceil (1 - \alpha)(t + 1) \rceil$ -th smallest of the past scores $\{s(\mathbf{x}_\tau, y_\tau)\}_{\tau=1}^t$. Its guarantee rests on exchangeability.

Lemma 2 (Rank uniformity). *If $S_1, \dots, S_{m+1}$ are exchangeable and a.s. distinct, then the rank of S_{m+1} among all $m+1$ values is uniform on $\{1, \dots, m+1\}$; hence for any integer k , $\mathbb{P}(S_{m+1} \leq S_{(k)}) \geq \frac{k}{m+1}$, where $S_{(k)}$ is the k -th smallest of $S_1, \dots, S_m$.*

Proof. Exchangeability makes all $(m+1)!$ orderings equally likely, so the rank R of S_{m+1} is uniform on $\{1, \dots, m+1\}$. The event $\{S_{m+1} \leq S_{(k)}\}$ contains $\{R \leq k\}$, and $\mathbb{P}(R \leq k) = k/(m+1)$. $\square$

Proposition 1 (Marginal coverage). *With $m = t$ calibration scores and $q_t = S_{(k)}$, $k = \lceil (1-\alpha)(t+1) \rceil$, the set (6) satisfies $\mathbb{P}(y_{t+1} \in \mathcal{C}_t(\mathbf{x}_{t+1})) \geq 1 - \alpha$.*

Proof. $y_{t+1} \in \mathcal{C}_t$ iff $s(\mathbf{x}_{t+1}, y_{t+1}) = S_{t+1} \leq q_t = S_{(k)}$. By Lemma 2, $\mathbb{P}(S_{t+1} \leq S_{(k)}) \geq k/(t+1) \geq 1 - \alpha$. $\square$

Code: the split-CP threshold. `run_plain_cp` returns exactly $S_{(k)}$ with $k = \lceil (1-\alpha)(t+1) \rceil$ (finite-sample correction `plus_one=True`); the min/max clamp caps k at r , returning the largest score rather than the strict $+\infty$ when $(1-\alpha)(r+1) > r$.

```
def run_plain_cp(calib_scores, alpha, plus_one=True):
    calib_scores = np.asarray(calib_scores, float).ravel()
    r = len(calib_scores)
    if r == 0:
        return np.inf
    if plus_one:
        k = int(np.ceil((1 - alpha) * (r + 1)))
    else:
        k = int(np.ceil((1 - alpha) * r))
    k = min(max(k, 1), r)
    return float(np.sort(calib_scores)[k - 1])
```

Score used here: the Gaussian NLL. With the GP predictive (3), the score is the negative predictive log-likelihood

$$s_t(y) = -\log p(y \mid \mathcal{D}_t, \mathbf{x}) = \underbrace{\frac{1}{2} \log(2\pi\sigma_t^2)}_{=: \text{nll_min}} + \frac{(y - \hat{y}_t)^2}{2\sigma_t^2}. \quad (7)$$

`standard_cp` is the online form of Proposition 1 for this score: at each t it sets q to the empirical $1 - \alpha$ quantile of *past* scores, inverts (7) to an interval (derived in §8, eq. (17)), and appends the new score to the calibration memory.

```
def standard_cp(y, mu, var, alpha=0.1):
    T = len(y)
    var = np.maximum(np.asarray(var, float), 1e-12)
    nll_min = 0.5 * np.log(2 * np.pi * var) # min score, at y = mu
    score = nll_min + (y - mu) ** 2 / (2 * var) # observed score s_t(Y_t)
    lo = np.full(T, np.nan); hi = np.full(T, np.nan)
    covered = np.zeros(T, bool); size = np.zeros(T); past = []
    for t in range(T):
        q = np.quantile(past, 1-alpha, method="higher") if past else score[t]
        rad = 2 * var[t] * (q - nll_min[t])
        if rad >= 0:
            h = np.sqrt(rad)
            lo[t], hi[t] = mu[t]-h, mu[t]+h
            size[t] = 2 * h
            covered[t] = score[t] <= q
            past.append(score[t])
    return dict(lo=lo, hi=hi, covered=covered, size=size)
```

The paper’s point: in the stream, calibration and test scores are *not* exchangeable (the model is trained on $\mathcal{D}_t$, so training scores are systematically smaller), and Proposition 1 can fail. Sections 8–10 restore a long-run guarantee by adapting q_t .

4 Scalable GP by random Fourier features (§III.A)

Assume $\kappa = \sigma_\theta^2 \bar{\kappa}$ with $\bar{\kappa}$ stationary and normalised, $\bar{\kappa}(\mathbf{x}, \mathbf{x}) = 1$.

Theorem 1 (Bochner). *A continuous stationary $\bar{\kappa}(\mathbf{x}, \mathbf{x}') = \bar{\kappa}(\mathbf{x} - \mathbf{x}')$ is positive definite iff it is the Fourier transform of a finite nonnegative measure. Normalising that measure to a density $\pi_{\bar{\kappa}}$ gives (paper eq. 5)*

$$\bar{\kappa}(\mathbf{x} - \mathbf{x}') = \int_{\mathbb{R}^d} \pi_{\bar{\kappa}}(\mathbf{v}) e^{j\mathbf{v}^\top(\mathbf{x} - \mathbf{x}')} d\mathbf{v} = \mathbb{E}_{\mathbf{v} \sim \pi_{\bar{\kappa}}} [e^{j\mathbf{v}^\top(\mathbf{x} - \mathbf{x}')}]. \quad (8)$$

Because $\bar{\kappa}$ is real and even, $\pi_{\bar{\kappa}}$ is symmetric, so the imaginary part of (8) integrates to zero and

$$\bar{\kappa}(\mathbf{x} - \mathbf{x}') = \mathbb{E}_{\mathbf{v}} [\cos(\mathbf{v}^\top(\mathbf{x} - \mathbf{x}'))] = \mathbb{E}_{\mathbf{v}} [\cos(\mathbf{v}^\top \mathbf{x}) \cos(\mathbf{v}^\top \mathbf{x}') + \sin(\mathbf{v}^\top \mathbf{x}) \sin(\mathbf{v}^\top \mathbf{x}')], \quad (9)$$

using $\cos(a - b) = \cos a \cos b + \sin a \sin b$. Draw $\mathbf{v}_1, \dots, \mathbf{v}_D \stackrel{\text{iid}}{\sim} \pi_{\bar{\kappa}}$ and define the $2D$ -dimensional random-feature map (paper eq. 6)

$$\phi_{\mathbf{v}}(\mathbf{x}) = \frac{1}{\sqrt{D}} [\sin(\mathbf{v}_1^\top \mathbf{x}), \cos(\mathbf{v}_1^\top \mathbf{x}), \dots, \sin(\mathbf{v}_D^\top \mathbf{x}), \cos(\mathbf{v}_D^\top \mathbf{x})]^\top. \quad (10)$$

Proposition 2 (Unbiasedness). $\mathbb{E}[\phi_{\mathbf{v}}(\mathbf{x})^\top \phi_{\mathbf{v}}(\mathbf{x}')] = \bar{\kappa}(\mathbf{x} - \mathbf{x}')$, and $\phi_{\mathbf{v}}(\mathbf{x})^\top \phi_{\mathbf{v}}(\mathbf{x}') \rightarrow \bar{\kappa}(\mathbf{x} - \mathbf{x}')$ a.s. as $D \rightarrow \infty$.

Proof. $\phi_{\mathbf{v}}(\mathbf{x})^\top \phi_{\mathbf{v}}(\mathbf{x}') = \frac{1}{D} \sum_{i=1}^D [\sin(\mathbf{v}_i^\top \mathbf{x}) \sin(\mathbf{v}_i^\top \mathbf{x}') + \cos(\mathbf{v}_i^\top \mathbf{x}) \cos(\mathbf{v}_i^\top \mathbf{x}')]$. Each summand has expectation $\bar{\kappa}(\mathbf{x} - \mathbf{x}')$ by (9); the average is therefore unbiased, and the strong law gives a.s. convergence. $\square$

Thus $\bar{\kappa}(\mathbf{x}, \mathbf{x}') \approx \phi_{\mathbf{v}}^\top(\mathbf{x}) \phi_{\mathbf{v}}(\mathbf{x}')$, and the parametric surrogate (paper eq. 7) is a Bayesian linear model

$$\check{f}(\mathbf{x}) = \phi_{\mathbf{v}}^\top(\mathbf{x}) \boldsymbol{\theta}, \quad \boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}_{2D}, \sigma_\theta^2 \mathbf{I}_{2D}), \quad (11)$$

whose induced covariance reproduces the target kernel: $\text{Cov}(\check{f}(\mathbf{x}), \check{f}(\mathbf{x}')) = \phi_{\mathbf{v}}^\top(\mathbf{x}) (\sigma_\theta^2 \mathbf{I}) \phi_{\mathbf{v}}(\mathbf{x}') = \sigma_\theta^2 \phi_{\mathbf{v}}^\top(\mathbf{x}) \phi_{\mathbf{v}}(\mathbf{x}') \approx \sigma_\theta^2 \bar{\kappa} = \kappa$.

Code: the feature map (10).

```
def rff_features(X, V):
    """X: (N, d), V: (d, D) -> Z: (N, 2D)."""
    D = V.shape[1]
    VX = X @ V
    return np.sqrt(1.0 / D) * np.concatenate([np.sin(VX), np.cos(VX)], axis=1)
```

Spectral densities and the samplers. The frequency law $\pi_{\bar{\kappa}}$ is the inverse transform of $\bar{\kappa}$. For the four kernels (1-D scale ℓ ; the code writes `length_scale =  $\ell$`):

Kernel $\bar{\kappa}(r)$	Spectral density $\pi_{\bar{\kappa}}(\omega)$	Sampling recipe (code)
RBF $e^{-r^2/2\ell^2}$	$\mathcal{N}(0, \ell^{-2})$	$\omega \sim \mathcal{N}(0, 1/\ell^2)$
Laplace $e^{- r /\ell}$	Cauchy(0, ℓ^{-1})	$\omega \sim \text{Cauchy}(0, 1/\ell)$
Matérn- ν	Student- $t_{2\nu}$ (scale ℓ^{-1})	$\omega = z/\sqrt{g}$, $z \sim \mathcal{N}(0, \ell^{-2})$, $g \sim \text{Gamma}(\nu, \frac{1}{\nu})$
Rational quad. (α)	Gamma mixture of $\mathcal{N}$	$\omega = z\sqrt{u}$, $z \sim \mathcal{N}(0, 1)$, $u \sim \text{Gamma}(\alpha, \frac{1}{\alpha\ell^2})$

The Matérn and RQ rows use the *Gaussian scale-mixture* representation: if $\omega \mid g \sim \mathcal{N}(0, \tau^2/g)$ and $g \sim \text{Gamma}(\nu, 1/\nu)$ (mean 1), the marginal of ω is Student- t with 2ν degrees of freedom; mixing $\mathcal{N}(0, u)$ over $u \sim \text{Gamma}$ yields the RQ spectrum. This is why the code draws a per-frequency mixing variable and rescales a Gaussian.

```
def sample_rbf(d, D, length_scale, rng): # RBF: omega ~ N(0, 1/l^2)
    return rng.normal(0.0, 1.0 / length_scale, size=(d, D))

def sample_laplacian(d, D, length_scale, rng): # exp(-|r|/l): omega ~ Cauchy(0, 1/l)
    return cauchy.rvs(loc=0.0, scale=1.0 / length_scale, size=(d, D), random_state=rng)

def sample_matern(d, D, length_scale, nu, rng): # Student-t_{2nu} via Gaussian scale-mixture
    g = gamma.rvs(a=nu, scale=1.0 / nu, size=D, random_state=rng)
    z = rng.normal(0.0, 1.0 / length_scale, size=(d, D))
    return z / np.sqrt(g)[None, :]

def sample_rq(d, D, length_scale, alpha, rng): # RQ = scale-mixture of RBFs
    u = gamma.rvs(a=alpha, scale=1.0 / (alpha * length_scale ** 2), size=D, random_state=rng)
    z = rng.normal(0.0, 1.0, size=(d, D))
    return z * np.sqrt(u)[None, :]
```

5 Weight-space posterior and online predictive (§III.A)

Stack $\phi_\tau = \phi_v(\mathbf{x}_\tau)$ into $\Phi_t = [\phi_1, \dots, \phi_t]^\top$. Model (11) with likelihood $y_\tau \mid \theta \sim \mathcal{N}(\phi_\tau^\top \theta, \sigma_n^2)$ is a conjugate Gaussian linear model; the posterior is Gaussian, $p(\theta \mid \mathcal{D}_t) = \mathcal{N}(\theta; \hat{\theta}_t, \Sigma_t)$.

Proposition 3 (Batch posterior).

$$\Sigma_t = \left(\sigma_n^{-2} \Phi_t^\top \Phi_t + \sigma_\theta^{-2} \mathbf{I} \right)^{-1}, \quad \hat{\theta}_t = \sigma_n^{-2} \Sigma_t \Phi_t^\top \mathbf{y}_t. \quad (12)$$

Proof. The log-posterior is $-\frac{1}{2\sigma_n^2} \|\mathbf{y}_t - \Phi_t \theta\|^2 - \frac{1}{2\sigma_\theta^2} \|\theta\|^2 + \text{const}$. Collecting quadratic and linear terms in θ gives precision $\sigma_n^{-2} \Phi_t^\top \Phi_t + \sigma_\theta^{-2} \mathbf{I}$ and linear coefficient $\sigma_n^{-2} \Phi_t^\top \mathbf{y}_t$; completing the square yields (12). $\square$

The predictive of y at a test $\mathbf{x}$ integrates θ out: with $\phi = \phi_v(\mathbf{x})$,

$$\hat{y}(\mathbf{x}) = \phi^\top \hat{\theta}_t, \quad \sigma^2(\mathbf{x}) = \phi^\top \Sigma_t \phi + \sigma_n^2, \quad (13)$$

the extra σ_n^2 being the observation noise. Equation (13) at $\mathbf{x}_{t+1}$ is the paper's eq. 8, written $\hat{y}_{t+1|t} = \phi^\top(\mathbf{x}_{t+1}) \hat{\theta}_t$ and $\sigma_{t+1|t}^2 = \phi^\top(\mathbf{x}_{t+1}) \Sigma_t \phi(\mathbf{x}_{t+1}) + \sigma_n^2$.

Code: batch weight-space solve. `gp_predict` implements (12)–(13) with $\Sigma_t = \sigma_n^2 \mathbf{A}^{-1}$, $\mathbf{A} = \Phi^\top \Phi + (\sigma_n^2 / \sigma_\theta^2) \mathbf{I}$: then $\hat{\theta}_t = \mathbf{A}^{-1} \Phi^\top \mathbf{y}$ and $\sigma^2 = \sigma_n^2 \phi^\top \mathbf{A}^{-1} \phi + \sigma_n^2$.

```
def gp_predict(Z_train, y_train, Z_test, sigma_n, sigma_f):
    M = Z_train.shape[1]
    A = Z_train.T @ Z_train + (sigma_n ** 2 / sigma_f ** 2) * np.eye(M)
    w = np.linalg.solve(A, Z_train.T @ y_train) # posterior mean weights
    mu = Z_test @ w
    AinvZt = np.linalg.solve(A, Z_test.T) # (M, N_test)
    var = sigma_n ** 2 * np.einsum('ij,ji->i', Z_test, AinvZt) + sigma_n ** 2
    return mu, var
```

The driver `run_online_gp` calls this on a growing training set (it therefore stores all past data — the non-memoryless variant, used for the mini-batch experiments).

6 The memoryless recursive update (§III.A, eqs. 10–11)

Absorbing one datum $(\mathbf{x}_{t+1}, y_{t+1})$ updates the posterior by Bayes' rule (paper eq. 10), $p(\boldsymbol{\theta} \mid \mathcal{D}_{t+1}) \propto p(\boldsymbol{\theta} \mid \mathcal{D}_t) \mathcal{N}(y_{t+1}; \boldsymbol{\phi}^\top \boldsymbol{\theta}, \sigma_n^2)$, again Gaussian. Write $\boldsymbol{\phi} = \boldsymbol{\phi}(\mathbf{x}_{t+1})$.

Theorem 2 (Recursive update, paper eq. 11). *With $\hat{y}_{t+1|t} = \boldsymbol{\phi}^\top \hat{\boldsymbol{\theta}}_t$ and $\sigma_{t+1|t}^2 = \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t \boldsymbol{\phi} + \sigma_n^2$,*

$$\hat{\boldsymbol{\theta}}_{t+1} = \hat{\boldsymbol{\theta}}_t + \sigma_{t+1|t}^{-2} \boldsymbol{\Sigma}_t \boldsymbol{\phi} (y_{t+1} - \hat{y}_{t+1|t}), \quad (14)$$

$$\boldsymbol{\Sigma}_{t+1} = \boldsymbol{\Sigma}_t - \sigma_{t+1|t}^{-2} \boldsymbol{\Sigma}_t \boldsymbol{\phi} \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t. \quad (15)$$

Proof. The updated precision is $\boldsymbol{\Sigma}_{t+1}^{-1} = \boldsymbol{\Sigma}_t^{-1} + \sigma_n^{-2} \boldsymbol{\phi} \boldsymbol{\phi}^\top$ (add one likelihood term to Proposition 3). Apply the Sherman–Morrison identity $(\mathbf{M} + \mathbf{u} \mathbf{u}^\top)^{-1} = \mathbf{M}^{-1} - \frac{\mathbf{M}^{-1} \mathbf{u} \mathbf{u}^\top \mathbf{M}^{-1}}{1 + \mathbf{u}^\top \mathbf{M}^{-1} \mathbf{u}}$ with $\mathbf{M} = \boldsymbol{\Sigma}_t^{-1}$, $\mathbf{u} = \sigma_n^{-1} \boldsymbol{\phi}$:

$$\boldsymbol{\Sigma}_{t+1} = \boldsymbol{\Sigma}_t - \frac{\sigma_n^{-2} \boldsymbol{\Sigma}_t \boldsymbol{\phi} \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t}{1 + \sigma_n^{-2} \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t \boldsymbol{\phi}} = \boldsymbol{\Sigma}_t - \frac{\boldsymbol{\Sigma}_t \boldsymbol{\phi} \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t}{\sigma_n^2 + \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t \boldsymbol{\phi}} = \boldsymbol{\Sigma}_t - \frac{\boldsymbol{\Sigma}_t \boldsymbol{\phi} \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t}{\sigma_{t+1|t}^2},$$

which is (15). For the mean, the conjugate update is $\hat{\boldsymbol{\theta}}_{t+1} = \boldsymbol{\Sigma}_{t+1}^{-1} (\boldsymbol{\Sigma}_t^{-1} \hat{\boldsymbol{\theta}}_t + \sigma_n^{-2} \boldsymbol{\phi} y_{t+1}) = \hat{\boldsymbol{\theta}}_t + \sigma_n^{-2} \boldsymbol{\Sigma}_{t+1} \boldsymbol{\phi} (y_{t+1} - \boldsymbol{\phi}^\top \hat{\boldsymbol{\theta}}_t)$, using $\boldsymbol{\Sigma}_{t+1} \boldsymbol{\Sigma}_t^{-1} = \mathbf{I} - \sigma_n^{-2} \boldsymbol{\Sigma}_{t+1} \boldsymbol{\phi} \boldsymbol{\phi}^\top$. Finally

$$\boldsymbol{\Sigma}_{t+1} \boldsymbol{\phi} = \boldsymbol{\Sigma}_t \boldsymbol{\phi} - \frac{\boldsymbol{\Sigma}_t \boldsymbol{\phi} (\boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t \boldsymbol{\phi})}{\sigma_{t+1|t}^2} = \boldsymbol{\Sigma}_t \boldsymbol{\phi} \frac{\sigma_{t+1|t}^2 - \boldsymbol{\phi}^\top \boldsymbol{\Sigma}_t \boldsymbol{\phi}}{\sigma_{t+1|t}^2} = \boldsymbol{\Sigma}_t \boldsymbol{\phi} \frac{\sigma_n^2}{\sigma_{t+1|t}^2},$$

so $\sigma_n^{-2} \boldsymbol{\Sigma}_{t+1} \boldsymbol{\phi} = \sigma_{t+1|t}^{-2} \boldsymbol{\Sigma}_t \boldsymbol{\phi}$, giving (14). $\square$

Each step costs $O((2D)^2)$ (a rank-one update of $\boldsymbol{\Sigma} \in \mathbb{R}^{2D \times 2D}$) and stores no past data. `run_online_gp_recursive` is (14)–(15) verbatim: `Sigma` starts at $\sigma_\theta^2 \mathbf{I}$ (prior), `s2` is $\sigma_{t+1|t}^2$, `gain` = $\boldsymbol{\Sigma}_t \boldsymbol{\phi} / \sigma_{t+1|t}^2$, and the symmetrisation line guards against round-off.

```
def run_online_gp_recursive(X_all, y_all, V, sigma_n, sigma_f, n_init=100):
    M = 2 * V.shape[1]
    theta = np.zeros(M)
    Sigma = (sigma_f ** 2) * np.eye(M) # prior theta ~ N(0, sigma_f^2 I)
    Z = rff_features(X_all, V); sn2 = sigma_n ** 2
    preds, truths, varis = [], [], []
    for t in range(len(X_all)):
        phi = Z[t]
        mu = phi @ theta
        Sphi = Sigma @ phi
        s2 = float(phi @ Sphi + sn2) # predictive variance sigma^2_{t+1|t}
        if t >= n_init:
            preds.append(mu); truths.append(y_all[t]); varis.append(s2)
        gain = Sphi / s2 # recursive update
        theta = theta + gain * (y_all[t] - mu)
        Sigma = Sigma - np.outer(Sphi, Sphi) / s2
        Sigma = 0.5 * (Sigma + Sigma.T) # keep symmetric
    return np.array(preds), np.array(truths), np.array(varis)
```

7 Online Bayes credible set (§III.A, eq. 9)

Using (13) at $\mathbf{x}_{t+1}$, the Bayes β -credible set is (paper eq. 9)

$$K_t^\beta(\mathbf{x}_{t+1}) = [\hat{y}_{t+1|t} - c_\beta \sigma_{t+1|t}, \hat{y}_{t+1|t} + c_\beta \sigma_{t+1|t}], \quad (16)$$

with c_β constant. `bayes_credible` uses $c_\beta = \Phi^{-1}(1 - \alpha/2)$ (so $\beta = 1 - \alpha$; $c_\beta \approx 1.645$ at $\alpha = 0.1$). Under mis-specification the fixed c_β makes (16) mis-covered — the motivation for a data-driven, time-varying multiplier next.

```
def bayes_credible(y, mu, var, alpha=0.1):
    c = norm.ppf(1 - alpha / 2) # fixed multiplier (1.645 @ alpha=0.1)
    sd = np.sqrt(np.maximum(var, 1e-12))
    return dict(lo=mu - c*sd, hi=mu + c*sd,
               covered=np.abs(y - mu) <= c*sd, size=2*c*sd)
```

8 Conformalised GP set and threshold update (§III.B)

Take the score (7). Inverting (6) gives a closed interval (paper eq. 12).

Proposition 4 (Set inversion). *For s_t in (7) and threshold q_t ,*

$$\mathcal{C}_t(\mathbf{x}_{t+1}) = \{y : s_t(y) \leq q_t\} = [\hat{y}_{t+1|t} - c_{t+1|t}\sigma_{t+1|t}, \hat{y}_{t+1|t} + c_{t+1|t}\sigma_{t+1|t}], \quad c_{t+1|t} = \sqrt{2q_t - \log(2\pi\sigma_{t+1|t}^2)}, \quad (17)$$

which is nonempty iff $q_t \geq \frac{1}{2} \log(2\pi\sigma_{t+1|t}^2)$, and is always bounded.

Proof. $s_t(y) \leq q_t \iff \frac{(y - \hat{y})^2}{2\sigma^2} \leq q_t - \frac{1}{2} \log(2\pi\sigma^2) \iff (y - \hat{y})^2 \leq 2\sigma^2 q_t - \sigma^2 \log(2\pi\sigma^2) = \sigma^2(2q_t - \log 2\pi\sigma^2)$. The right side is ≥ 0 iff $q_t \geq \frac{1}{2} \log(2\pi\sigma^2)$; taking square roots gives half-width $\sigma\sqrt{2q_t - \log 2\pi\sigma^2} = c_{t+1|t}\sigma$. Because $s_t(y) \rightarrow \infty$ as $|y| \rightarrow \infty$, the sublevel set is bounded (contrast a residual score, whose set can be all of $\mathbb{R}$). $\square$

Unlike the constant c_β of (16), here $c_{t+1|t}$ "changes over time, adapting to new data samples." The threshold is driven by coverage feedback (paper eq. 13):

$$q_{t+1} = q_t + \eta_t(\mathbb{I}(y_{t+1} \notin \mathcal{C}_t(\mathbf{x}_{t+1})) - \alpha). \quad (18)$$

Proposition 5 (eq. 13 is online sub-gradient descent on the pinball loss, eq. 14). *Let $\rho_{1-\alpha}(u) = (1 - \alpha) \max\{u, 0\} + \alpha \max\{-u, 0\}$ and $u_t = s_t(\mathbf{x}_{t+1}, y_{t+1}) - q_t$. Then a step $q_{t+1} = q_t - \eta_t \partial_q \rho_{1-\alpha}(u_t)$ equals (18).*

Proof. For $u \neq 0$, $\rho'_{1-\alpha}(u) = (1 - \alpha)$ if $u > 0$ and $-\alpha$ if $u < 0$. Since $u_t = s_t - q_t$, $\partial_q \rho = -\rho'(u_t)$, i.e. $-(1 - \alpha)$ when $s_t > q_t$ (a miss, $y_{t+1} \notin \mathcal{C}_t$) and $+\alpha$ when $s_t < q_t$ (covered). Hence $q_{t+1} = q_t - \eta_t \partial_q \rho$ equals $q_t + \eta_t(1 - \alpha)$ on a miss and $q_t - \eta_t\alpha$ when covered — exactly $q_t + \eta_t(\mathbb{I}(\text{miss}) - \alpha)$. $\square$

Code. `online_conformal` computes the score (7), inverts it (`rad= 2σ2(q−nll_min)`), `h= ct+1|tσt+1|t`; empty set when `rad< 0`), records coverage, and applies (18) in the final line. The `step` argument selects η_t (§9).

```
def online_conformal(y, mu, var, alpha=0.1, step="varying", c=1.0, eps=0.1,
                    W=15, r=100, q1=None):
    y = np.asarray(y, float); T = len(y)
    var = np.maximum(np.asarray(var, float), 1e-12)
    nll_min = 0.5 * np.log(2 * np.pi * var)
    score = nll_min + (y - mu) ** 2 / (2 * var)
    if q1 is None:
        q1 = float(np.median(nll_min) + chi2.ppf(1.0 - alpha, df=1) / 2)
    q = q1
    lo = np.full(T, np.nan); hi = np.full(T, np.nan)
```

```

size = np.zeros(T); covered = np.zeros(T, bool); q_path = np.empty(T)
resets, win = [], []; tau = 1; prev_avg = None; rises = 0; a = 0.5 + eps
for t in range(T):
    q_path[t] = q
    rad = 2 * var[t] * (q - nll_min[t]) # invert the NLL score
    if rad >= 0.0:
        h = np.sqrt(rad)
        lo[t], hi[t] = mu[t] - h, mu[t] + h
        size[t] = 2 * h
    covered[t] = score[t] <= q
    if step == "varying":
        win.append(size[t])
        if len(win) > W: win.pop(0)
        if len(win) == W: # set-size change detector
            avg = np.mean(win)
            if (prev_avg is not None and avg > prev_avg): rises += 1
            else: rises = 0
            prev_avg = avg
            if rises >= r:
                tau = 1; rises = 0; resets.append(t)
        eta = c * tau ** (-a); tau += 1
    elif step == "decaying":
        eta = c * (t + 1) ** (-a)
    else:
        eta = float(step) # fixed constant eta
    q = q + eta * ((0.0 if covered[t] else 1.0) - alpha) # eq. (13)
return dict(lo=lo, hi=hi, size=size, covered=covered, q_path=q_path, resets=resets)

```

9 Learning-rate schedules and change-point reset (§III.B)

Three regimes for η_t , all in the branch above (with $a = \frac{1}{2} + \varepsilon$):

- **Constant** $\eta_t = \eta$ (`step=<float>`): asymptotic coverage, but the paper notes it "will yield high variability in the instantaneous coverage."
- **Decaying** $\eta_t = c(t+1)^{-a}$ (`step="decaying"`): valid under stationary or slow drift; the experiments use $\eta_t = t^{-3/5}$ ($\varepsilon = 0.1$).
- **Varying** $\eta_t = c\tau^{-a}$ with τ the slots since the last reset (`step="varying"`): the set size $|\mathcal{C}_t| = 2c_{t+1|t}\sigma_{t+1|t}$ is averaged over a window of W slots, and "if the average prediction set size increases over r consecutive steps, we will declare a distributional shift and then reset the learning rate" ($\tau \leftarrow 1$). Defaults $W = 15$, $r = 100$.

10 Long-run coverage guarantee (§III.B, eq. 15)

Theorem 3 (paper eq. 15). *If $s_t : \mathcal{X} \times \mathcal{Y} \rightarrow [0, B]$ and $q_0 \in [0, B]$, then for any data sequence and any positive $\{\eta_t\}$,*

$$\left| \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{I}(y_{t+1} \in \mathcal{C}_t(\mathbf{x}_{t+1})) - (1 - \alpha) \right| \leq \frac{C N_T}{T}, \quad C = \frac{2(B + \max_t \eta_t)}{\min_t \eta_t}, \quad N_T = \sum_{\tau=1}^T \mathbb{I}(\eta_\tau > \eta_{\tau-1}). \quad (19)$$

Derivation (constant η , then the reset correction). Let $\text{err}_t = \mathbb{I}(y_{t+1} \notin \mathcal{C}_t)$, so (18) is $q_{t+1} = q_t + \eta_t(\text{err}_t - \alpha)$. First take $\eta_t \equiv \eta$. Telescoping,

$$q_T - q_0 = \eta \sum_{t=0}^{T-1} (\text{err}_t - \alpha) \implies \frac{1}{T} \sum_{t=0}^{T-1} (\text{err}_t - \alpha) = \frac{q_T - q_0}{\eta T}.$$

It remains to bound $|q_T - q_0|$. A key monotonicity of the update keeps q_t in a bounded band: if $q_t > B$ then $s_t \leq B < q_t$ so the point is covered ($\text{err}_t = 0$) and $q_{t+1} = q_t - \eta\alpha \leq q_t$; if $q_t < 0$ then $s_t \geq 0 > q_t$ so it is a miss and $q_{t+1} = q_t + \eta(1 - \alpha) \geq q_t$. Hence, starting from $q_0 \in [0, B]$, $q_t \in [-\eta\alpha, B + \eta(1 - \alpha)] \subseteq [-\eta, B + \eta]$ for all t , so $|q_T - q_0| \leq B + \eta$. Therefore

$$\left| \frac{1}{T} \sum_{t=0}^{T-1} (\text{err}_t - \alpha) \right| \leq \frac{B + \eta}{\eta T},$$

and since $\frac{1}{T} \sum \mathbb{I}(y_{t+1} \in \mathcal{C}_t) - (1 - \alpha) = -(\frac{1}{T} \sum \text{err}_t - \alpha)$, this is (19) with $N_T = 1$ (no rate changes). For a general positive schedule, rewrite the telescoped sum with the varying η_t and split it at the steps where η_t increases: on any block where η_t is non-increasing the same clipping argument bounds the block's contribution by $(B + \max_t \eta_t) / \min_t \eta_t$, and there are N_T such increase events, giving the $C N_T$ numerator. Full accounting is in Angelopoulos, Barber & Bates (2024) and Xu et al. (2024). $\square$

The practical reading, quoted from the paper: "as long as the step size does not decay too quickly, and the number of resets N_T is sublinear in T , OS-GP-CP can achieve long-run coverage convergence." In code, the returned `resets` list is the set of increase events counted by N_T ; the varying schedule aims to keep $|\text{resets}| = N_T = o(T)$.

11 Hyperparameters and experimental setup (§IV)

The kernel magnitude σ_θ^2 , lengthscale ℓ^2 and noise σ_n^2 are fit by maximising the GP log marginal likelihood on the first 100 points,

$$\log p(\mathbf{y} \mid \mathbf{X}, \vartheta) = -\frac{1}{2} \mathbf{y}^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K} + \sigma_n^2 \mathbf{I}| - \frac{t}{2} \log 2\pi, \quad (20)$$

then frozen; $D = 200$ spectral features and an RBF kernel are used. `fit_hyperparameters` optimises (20) through scikit-learn's `ConstantKernel*RBF + WhiteKernel` and returns $(\ell, \sigma_\theta, \sigma_n)$.

```
def fit_hyperparameters(X, y, n0=100):
    k = (ConstantKernel(1.0, (1e-3, 1e3)) * RBF(1.0, (1e-2, 1e2))) + WhiteKernel(0.1, (1e-6, 1e1))
    gpr = GaussianProcessRegressor(kernel=k, alpha=0.0, n_restarts_optimizer=2).fit(X[:n0], y[:n0])
    p = gpr.kernel_.get_params()
    return (float(p["k1__k2__length_scale"]), # length_scale
            float(np.sqrt(p["k1__k1__constant_value"])), # sigma_f (= sigma_theta)
            float(np.sqrt(max(p["k2__noise_level"], 1e-6)))) # sigma_n
```

12 Extension in the code: the kernel ensemble

The manuscript's experiments use a single RBF kernel. The code additionally forms a four-kernel ensemble by causal Bayesian model averaging (BMA); this is an add-on, not part of the paper's

stated method. Given per-model Gaussian predictives $\mathcal{N}(\mu_i, \sigma_i^2)$ with weights w_i , the mixture’s first two moments follow from the law of total variance,

$$\mu_{\text{ens}} = \sum_i w_i \mu_i, \quad \sigma_{\text{ens}}^2 = \sum_i w_i (\sigma_i^2 + \mu_i^2) - \left(\sum_i w_i \mu_i \right)^2, \quad (21)$$

and the BMA weights update multiplicatively by the marginal likelihood, $w_i \leftarrow w_i p_i(y_t) / \sum_j w_j p_j(y_t)$, done in log-space (weights for t use only points $< t$, so there is no leakage). `gaussian_loglik` supplies $\log p_i(y_t)$ and `ensemble` carries (21) plus the weight recursion; the moment-matched $(\mu_{\text{ens}}, \sigma_{\text{ens}}^2)$ then feeds the conformal layer of §8.

```
def gaussian_loglik(y, mu, var):
    var = np.maximum(var, 1e-12)
    return -0.5 * (np.log(2 * np.pi * var) + (y - mu) ** 2 / var)

def ensemble(preds, varis, logliks, init_weights=None):
    n_models, T = preds.shape
    w = (np.full(n_models, 1.0 / n_models) if init_weights is None
         else np.asarray(init_weights, float))
    ens_mu = np.zeros(T); ens_var = np.zeros(T); weight_hist = np.zeros((T, n_models))
    for t in range(T):
        weight_hist[t] = w
        mu_t = w @ preds[:, t] # mixture mean
        second = w @ (varis[:, t] + preds[:, t] ** 2) # E[y^2] under the mixture
        ens_mu[t] = mu_t; ens_var[t] = second - mu_t ** 2 # law of total variance
        logw = np.log(w + 1e-300) + logliks[:, t] # BMA weight update (log-space)
        logw -= logw.max(); w = np.exp(logw); w /= w.sum()
    return ens_mu, ens_var, weight_hist
```

13 Equation-to-code map

Paper	Object	Function
(1)–(3)	exact GP joint / predictive	Lemma 1; realised via features (§5)
(4)	conformal set + split-CP quantile	<code>run_plain_cp</code> , <code>standard_cp</code>
(5)	spectral density / Bochner	<code>sample_rbf</code> / <code>laplacian</code> / <code>matern</code> / <code>rq</code>
(6)	random-feature map ϕ_v	<code>rff_features</code>
(7)	parametric model + weight prior	<code>run_online_gp_recursive</code> (prior $\sigma_\theta^2 \mathbf{I}$)
(8)	online predictive mean/variance	<code>run_online_gp_recursive</code> , <code>gp_predict</code>
(9)	Bayes credible set	<code>bayes_credible</code>
(10)–(11)	recursive weight-posterior update	<code>run_online_gp_recursive</code> (Thm. 2)
(12)	conformalised set, $c_{t+1 t}$	<code>online_conformal</code> (Prop. 4)
(13)–(14)	threshold feedback update	<code>online_conformal</code> (Prop. 5)
schedules	constant/decaying/varying η_t	<code>online_conformal</code> (<code>step</code> , <code>resets</code>)
(15)	coverage theorem, N_T	Thm. 3; <code>resets</code> are the N_T events
§IV	marginal-likelihood fit	<code>fit_hyperparameters</code>
—	kernel ensemble (extension)	<code>gaussian_loglik</code> , <code>ensemble</code>